Compare RFSI method vs RF (ignoring locations) method and evaluate performance

Narin Polat Öztürk

Task: Use the meuse data set and focus on "zinc" variable; compare performance of using RFSI method vs simple RF using 5–fold cross-validation: "Is there a difference in accuracy between the two methods?" try adding new covariate layers such as the inundated water and similar and see if this changes results.

1. Load the required packages

First, install the packages if they are not already installed.

The packeages are used for the following purposes:

- sp: provides the Meuse data set.
- meteo: creates nearest-neighbour variables required for RFSI.
- ranger: fits Random Forest models.
- ggplot2: creates figures.
- dplyr and tidyr: data manipulation.
- terra: handling additional raster covariates.

```
# Libraries
library(sp)
library(meteo)
library(ranger)
library(ggplot2)
library(dplyr)
library(tidyr)
library(terra)
```

2. Load the Meuse data

The Meuse data set contains heavy-metal concentrations measured in the floodplain of the Meuse River.

```
# Downloading muse dataset
data(meuse, package = "sp")
data(meuse.grid, package = "sp")
```

3. Define the baseline covariates

The same environmental covariates should be supplied to both RF and RFSI so that the comparison is fair.

```
# Defining the covars
covars <- c("dist", "ffreq", "soil")
names(meuse.grid)
```

```
[1] "x"      "y"      "part.a" "part.b" "dist"   "soil"   "ffreq"
```

```
meuse$ffreq <- factor(meuse$ffreq)
meuse$soil <- factor(meuse$soil)

meuse.grid$ffreq <- factor(meuse.grid$ffreq, levels = levels(meuse$ffreq))
meuse.grid$soil <- factor(meuse.grid$soil, levels = levels(meuse$soil))
```

The baseline Random Forest therefore represents a model that uses environmental covariates but does not explicitly use geographical coordinates or neighbouring zinc observations.

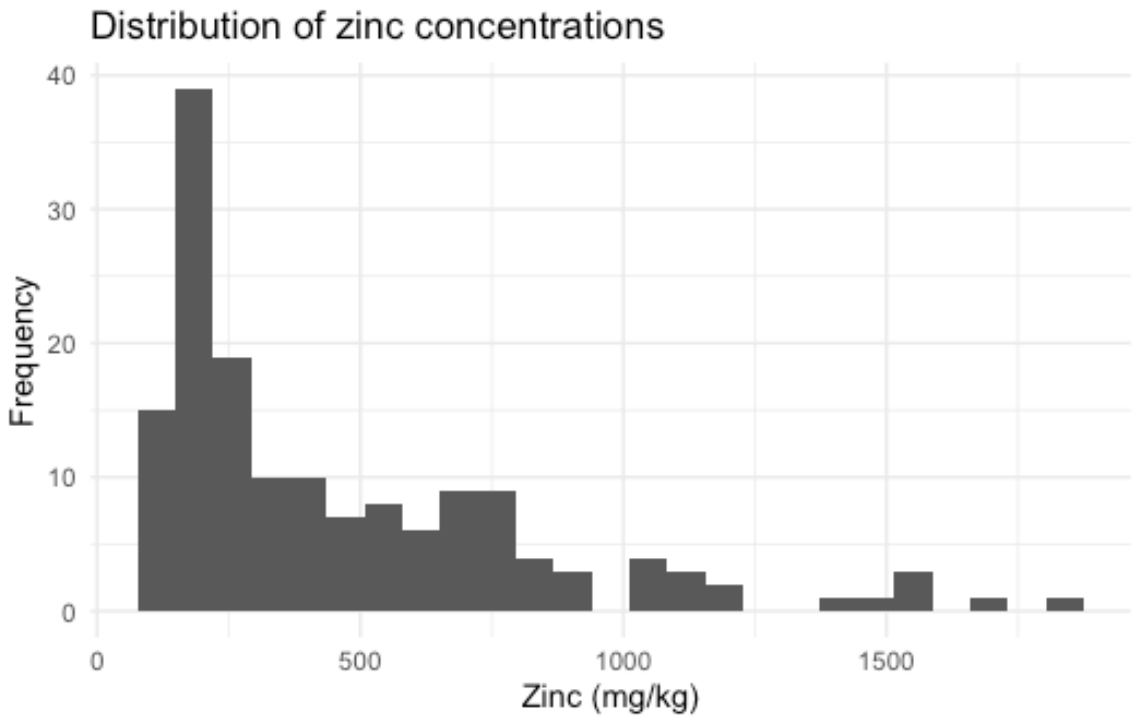
4. Explore the distribution of zinc

Before fitting the models, inspect the response variable.

```
# Distribution of zinc
summary(meuse$zinc)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  113.0   198.0   326.0   469.7   674.5  1839.0
```

```
ggplot(meuse, aes(x = zinc)) +
  geom_histogram(bins = 25) +
  theme_minimal() +
  labs(x = "Zinc (mg/kg)",
       y = "Frequency",
       title = "Distribution of zinc concentrations")
```



Zinc concentrations are expected to show a right-skewed distribution. For the initial comparison, the models can be fitted directly to the original zinc values. A log-transformed version can later be tested as a sensitivity analysis

5. Create 5-fold cross-validation

The observations are divided into five approximately equal groups.

```
# 5-fold cross validation
set.seed(42)

n <- nrow(meuse)

fold_id <- sample(rep(1:5, length.out = n))
table(fold_id)
```

```
fold_id
 1  2  3  4  5
31 31 31 31 31
```

The same folds must be used for both RF and RFSI.

This is important because otherwise differences in model performance could partly result from different train-validation partitions rather than from the modelling method itself.

6. Fit the baseline Random Forest

```
# Random Forest (RF)
train_id <- which(fold_id != 1)
test_id  <- which(fold_id == 1)
# training
train <- meuse[train_id, ]
# validation
test <- meuse[test_id, ]
# RF model
rf_model <- ranger(
  zinc ~ dist + ffreq + soil,
  data = train,
  num.trees = 500,
  seed = 42
)

# prediction
rf_pred <- predict(rf_model, data = test)$predictions

head(rf_pred)
```

```
[1] 317.9252 305.2056 917.5963 917.5963 976.1244 308.3656
```

It also does not use zinc concentrations measured at neighbouring sampling locations. Therefore, this represents the RF ignoring locations method required in the assignment.

7. Construct the spatial predictors for RFSI

The main difference between standard RF and RFSI is that RFSI introduces information from neighbouring observations.

```
# Random Forest Spatial Interpolation (RFS)
nearest_train <- near.obs(
  locations = train,
  locations.x.y = c("x", "y"),
  observations = train,
  observations.x.y = c("x", "y"),
  obs.col = "zinc",
  n.obs = 10,
  rm.dupl = TRUE
)
rm.dupl = TRUE
head(nearest_train)
```

```
      dist1    dist2    dist3    dist4    dist5    dist6    dist7    dist8
1  70.83784 118.8486 252.0496 258.3215 259.2393 336.4342 373.4836 380.1894
2  70.83784 141.5662 195.0103 234.4014 266.0113 282.8516 311.0820 328.8678
```

```
3 118.84864 141.5662 143.1712 167.0000 222.0811 323.5135 326.4016 345.8916
4 143.17123 175.1713 242.1570 259.2393 282.8516 296.7861 322.8188 324.4996
5 182.83326 242.1570 250.4496 275.8623 331.2718 356.8669 377.3301 391.7167
6 138.17742 162.9264 167.0000 174.2010 175.1713 212.8215 228.7794 234.4014
     dist9   dist10 obs1 obs2 obs3 obs4 obs5 obs6 obs7 obs8 obs9 obs10
1 399.6561 437.9372 1141  640  406  346  257 1096  504  347  279  1032
2 356.3495 367.9253 1022  640  406  346 1096  257  504  347  279  1032
3 351.9730 356.8669 1022 1141  257  346  406  347  279  504 1096   281
4 347.3514 402.5730  640  346  281 1022 1141  406  183  279  347   504
5 410.3584 421.0760  183  257  346  279  347  640  406  326  218   504
6 250.4496 258.3215  406  279  640  347  257  183  504 1141  281  1022
```

8. Calculate spatial predictors for the validation data

This is the most important part of the cross-validation procedure. For validation locations, neighbouring observations must be searched only within the training data.

```
# Calculating neasrest validation points
observations = meuse
nearest_test <- near.obs(
  locations = test,
  locations.x.y = c("x", "y"),
  observations = train,
  observations.x.y = c("x", "y"),
  obs.col = "zinc",
  n.obs = 10,
  rm.dupl = TRUE
)
observations = train
```

This prevents validation zinc concentrations from being used as predictors.

If all Meuse observations were used before cross-validation, the validation observations could become neighbours of one another and information would leak from the validation set into the model.

That would artificially improve the apparent performance of RFSI.

9. Combine the environmental and spatial predictors

```
# Determining the RFSI datasets
#training
train_rfsi <- cbind(train, nearest_train)
# validation
test_rfsi <- cbind(test, nearest_test)
# control
names(train_rfsi)
```

```
 [1] "x"       "y"       "cadmium" "copper"  "lead"    "zinc"    "elev"
 [8] "dist"    "om"      "ffreq"   "soil"    "lime"    "landuse" "dist.m"
[15] "dist1"   "dist2"   "dist3"   "dist4"   "dist5"   "dist6"   "dist7"
[22] "dist8"   "dist9"   "dist10"  "obs1"    "obs2"    "obs3"    "obs4"
[29] "obs5"    "obs6"    "obs7"    "obs8"    "obs9"    "obs10"
```

10. Create the RFSI formula

```
# Creating the RFSI formula
dist_vars <- paste0("dist", 1:10)
obs_vars <- paste0("obs", 1:10)
rfsi_vars <- c(covars, dist_vars, obs_vars)
rfsi_formula <- reformulate(rfsi_vars, response = "zinc")
rfsi_formula
```

```
zinc ~ dist + ffreq + soil + dist1 + dist2 + dist3 + dist4 +
    dist5 + dist6 + dist7 + dist8 + dist9 + dist10 + obs1 + obs2 +
    obs3 + obs4 + obs5 + obs6 + obs7 + obs8 + obs9 + obs10
```

11. Fit the RFSO model

Fit Random Forest using both environmental covariates and spatial neighbour variables.

```
# Learning RFSI model
rfsi_model <- ranger(
  formula = rfsi_formula,
  data = train_rfsi,
  num.trees = 500,
  seed = 42
)
rfsi_pred <- predict(rfsi_model, data = test_rfsi)$predictions

head(data.frame(
    observed = test$zinc,
    RF = rf_pred,
    RFSI = rfsi_pred
))
```

```
  observed       RF     RFSI
1      269 317.9252 275.3214
2      189 305.2056 276.0687
3      606 917.5963 799.9307
4      711 917.5963 738.8492
5      735 976.1244 842.1629
6      180 308.3656 226.1580
```

At this stage, predictions from both approaches for Fold 1.

12. Run the complete 5-fold cross-validation

```
# 5 fold cycle process
cv_results <- data.frame()
for (fold in 1:5) {

  # ----------------------
  # Training / validation
  # ----------------------

  train_id <- which(fold_id != fold)
  test_id  <- which(fold_id == fold)

  train <- meuse[train_id, ]
  test  <- meuse[test_id, ]


  # ============================================================
  # RF
  # ============================================================

  rf_model <- ranger(
    zinc ~ dist + ffreq + soil,
    data = train,
    num.trees = 500,
    seed = 42 + fold
  )

  rf_pred <- predict(rf_model, data = test)$predictions


  # ============================================================
  # RFSI
  # ============================================================

  nearest_train <- near.obs(
    locations = train,
    locations.x.y = c("x", "y"),
    observations = train,
    observations.x.y = c("x", "y"),
    obs.col = "zinc",
    n.obs = 10,
    rm.dupl = TRUE
  )

  nearest_test <- near.obs(
    locations = test,
```

```
    locations.x.y = c("x", "y"),
    observations = train,
    observations.x.y = c("x", "y"),
    obs.col = "zinc",
    n.obs = 10,
    rm.dupl = TRUE
  )

  train_rfsi <- cbind(train, nearest_train)
  test_rfsi <- cbind(test, nearest_test)

  rfsi_model <- ranger(
    formula = rfsi_formula,
    data = train_rfsi,
    num.trees = 500,
    seed = 42 + fold
  )

  rfsi_pred <- predict(rfsi_model, data = test_rfsi)$predictions


  # ============================================================

  # ============================================================

  fold_results <- data.frame(
    fold = fold,
    observed = test$zinc,
    RF = rf_pred,
    RFSI = rfsi_pred
  )

  cv_results <- rbind(cv_results, fold_results)
}
```

13. Calculate RMSE

RMSE measures the average magnitude of prediction errors while giving relatively high weight to large errors.

```
# Calculation the RMSE
rmse <- function(obs, pred) {
  sqrt(mean((obs - pred)^2))
}
# RF
rmse_rf <- rmse(cv_results$observed, cv_results$RF)
# RFSI
rmse_rfsi <- rmse(cv_results$observed,cv_results$RFSI)
```

14. Calculate MAE

MAE represent the average absolute prediction error.

```
# calculating MAE
mae <- function(obs, pred) {
  mean(abs(obs - pred))
}

mae_rf <- mae(cv_results$observed,cv_results$RF)
mae_rfsi <- mae(cv_results$observed, cv_results$RFSI)
```

15. Calculate R2

R^2 describes how closely predicted and observed values are associated.

```
# Calculating R2
r2 <- function(obs, pred) {
  cor(obs, pred)^2
}

r2_rf <- r2(cv_results$observed, cv_results$RF)
r2_rfsi <- r2(cv_results$observed, cv_results$RFSI)
```

Higher R^2 indicates better agreement between observed and predicted values.

16. Create the performance table

Combine the metrics into one table.

```
# Result figures
performance <- data.frame(
  Model = c("RF", "RFSI"),
  RMSE = c(rmse_rf, rmse_rfsi),
  MAE = c(mae_rf, mae_rfsi),
  R2 = c(r2_rf, r2_rfsi)
)
performance
```

```
  Model     RMSE      MAE        R2
1    RF 223.8648 160.7594 0.6403105
2  RFSI 229.6631 159.4370 0.6210794
```

17. Calculate the relative improvement of RFSI

The percentage reduction in RMSE can make the comparison easier to interpret.

```
# Calculating the percentage improvement in the RFSI.
```

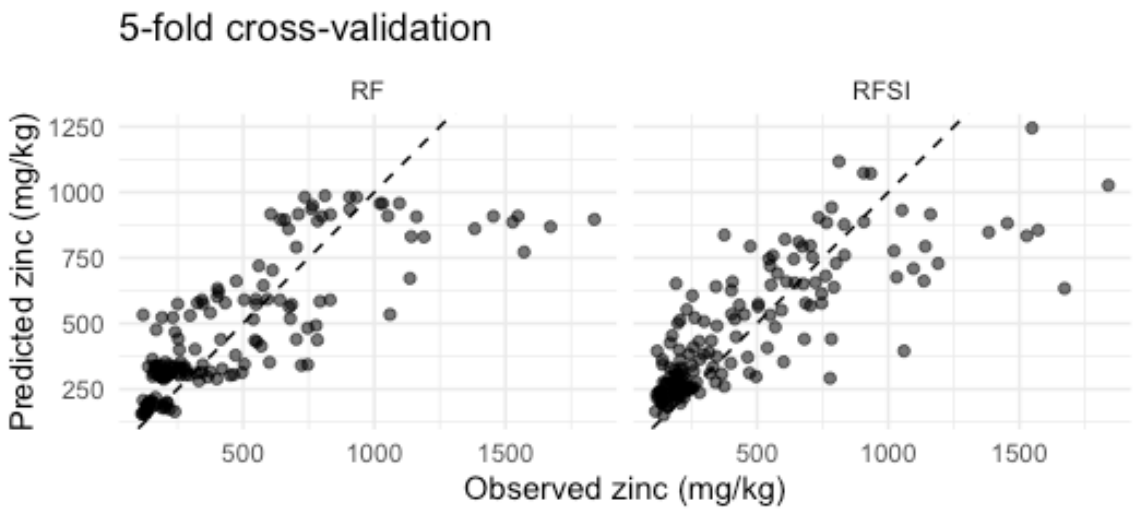
```
rmse_improvement <- 100 * (rmse_rf - rmse_rfsi) / rmse_rf
rmse_improvement
```

```
[1] -2.590116
```

If the value is negative, standart RF performed better.

18. Plot observed versus predicted values

```
# 18.Observed-predicted plot
plot_data <- cv_results |>
  pivot_longer(
    cols = c(RF, RFSI),
    names_to = "model",
    values_to = "predicted"
  )
ggplot(plot_data, aes(x = observed, y = predicted)) +
  geom_point(alpha = 0.6) +
  geom_abline(intercept = 0, slope = 1, linetype = 2) +
  facet_wrap(~ model) +
  coord_equal() +
  theme_minimal() +
  labs(
    x = "Observed zinc (mg/kg)",
    y = "Predicted zinc (mg/kg)",
    title = "5-fold cross-validation"
  )
```

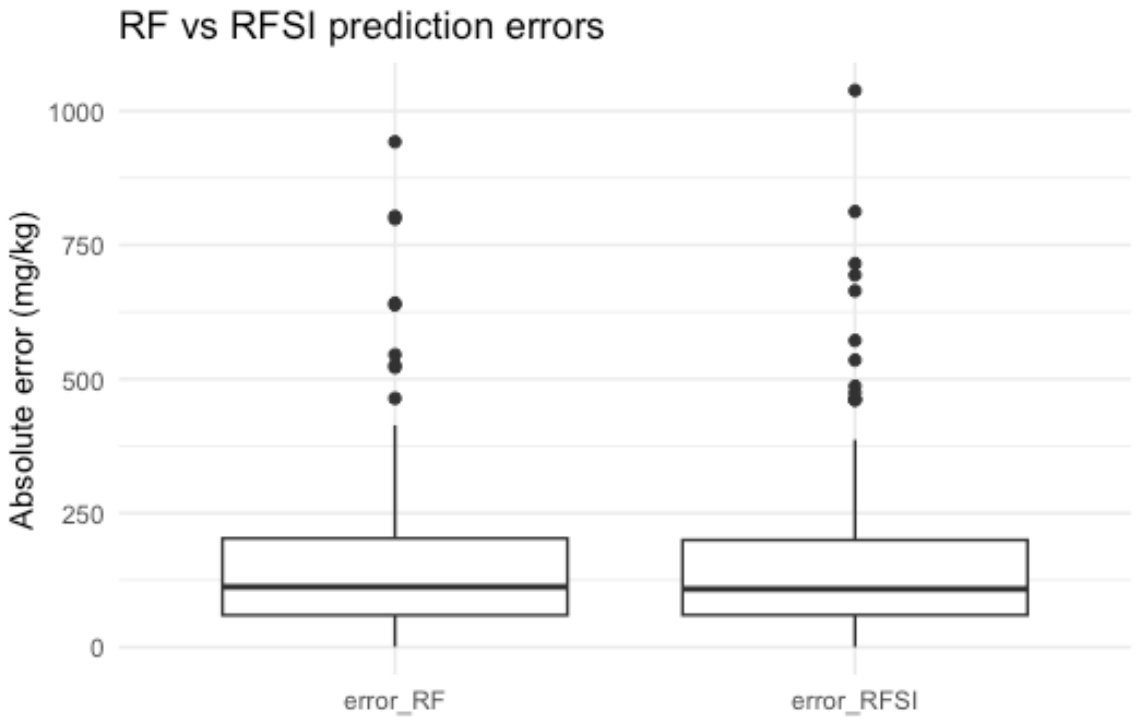


Points closer to the 1:1 line indicate better predictions.

19. Compare absolute prediction errors

```
# Comparing the errors
error_data <- cv_results |>
  mutate(
    error_RF = abs(observed - RF),
    error_RFSI = abs(observed - RFSI)
  )
error_long <- error_data |>
  select(error_RF, error_RFSI) |>
  pivot_longer(everything(), names_to = "model", values_to = "absolute_error")

ggplot(error_long, aes(x = model, y = absolute_error)) +
  geom_boxplot() +
  theme_minimal() +
  labs(
    x = NULL,
    y = "Absolute error (mg/kg)",
    title = "RF vs RFSI prediction errors"
  )
```



This figure shows whether RFSI generally produces smaller errors than standart RF.

20. Compare errors statistically

Because both models predict exactly the same validation observations, their errors are paired.

```
# Statistical difference between two models
wilcox.test(error_data$error_RF, error_data$error_RFSI, paired = TRUE)
```

```
    Wilcoxon signed rank test with continuity correction

data:  error_data$error_RF and error_data$error_RFSI
V = 6286, p-value = 0.6675
alternative hypothesis: true location shift is not equal to 0
```

However, this result should be interpreted cautiously because spatial observations are not necessarily statistically independent. Therefore, the primary evaluation should still rely on: RMSE MAE R^2 consistency among the five folds rather than only on the p-value.

21. Calculate performance separately for each fold

It is useful to check whether one method consistently performs better or whether the overall difference is driven by only one cross-validation fold.

```r
# Looking at the performance on a fold basis

fold_metrics <- cv_results |>
group_by(fold) |>
  summarise(
    RMSE_RF = sqrt(mean((observed - RF)^2)),
    RMSE_RFSI = sqrt(mean((observed - RFSI)^2)),
    MAE_RF = mean(abs(observed - RF)),
    MAE_RFSI = mean(abs(observed - RFSI))
  )
fold_metrics
```

```
# A tibble: 5 × 5
   fold RMSE_RF RMSE_RFSI MAE_RF MAE_RFSI
  <int>   <dbl>     <dbl>  <dbl>    <dbl>
1     1    227.      253.   172.     165.
2     2    233.      227.   157.     155.
3     3    190.      177.   130.     134.
4     4    199.      233.   157.     180.
5     5    262.      250.   189.     163.
```

22. Fit the final Random Forest model

Cross-validation is used to evaluate model performance.

After evaluation, a final model can be trained using all available observations to create the prediction map.

```r
# Last model
rf_final <- ranger(
  zinc ~ dist + ffreq + soil,
  data = meuse,
  num.trees = 500,
  seed = 42
)
meuse.grid$zinc_RF <- predict(rf_final, data = meuse.grid)$predictions
```

23. Fit the final RFSI model

First calculate nearest neighbours among the complete set of observations.

```r
# Fit the final RFSI model
nearest_meuse <- near.obs(
  locations = meuse,
  locations.x.y = c("x", "y"),
  observations = meuse,
  observations.x.y = c("x", "y"),
```

```
  obs.col = "zinc",
  n.obs = 10,
  rm.dupl = TRUE
)

meuse_rfsi <- cbind(meuse, nearest_meuse)

rfsi_final <- ranger(
  formula = rfsi_formula,
  data = meuse_rfsi,
  num.trees = 500,
  seed = 42
)
```

24. Calculate nearest neighbours for the prediction grid

For every grid cell, identify the ten nearest Meuse observations.

```
# Calculate nearest neighbours for the prediction grid
nearest_grid <- near.obs(
  locations = meuse.grid,
  locations.x.y = c("x", "y"),
  observations = meuse,
  observations.x.y = c("x", "y"),
  obs.col = "zinc",
  n.obs = 10,
  rm.dupl = TRUE
)
grid_rfsi <- cbind(meuse.grid, nearest_grid)
meuse.grid$zinc_RFSI <- predict(rfsi_final, data = grid_rfsi)$predictions
```

25. Calculate differences between the two prediction maps

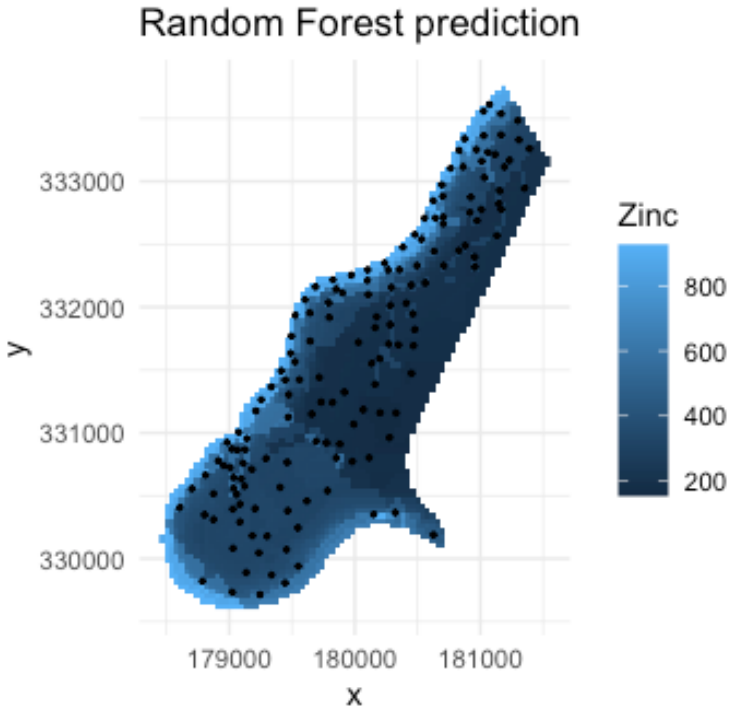
The assignment specifically asks for a visual comparison of prediction differences.

```
# Calculate differences between the two prediction maps
meuse.grid$diff <- meuse.grid$zinc_RFSI - meuse.grid$zinc_RF
```

26. Plot the Random Forest prediction map

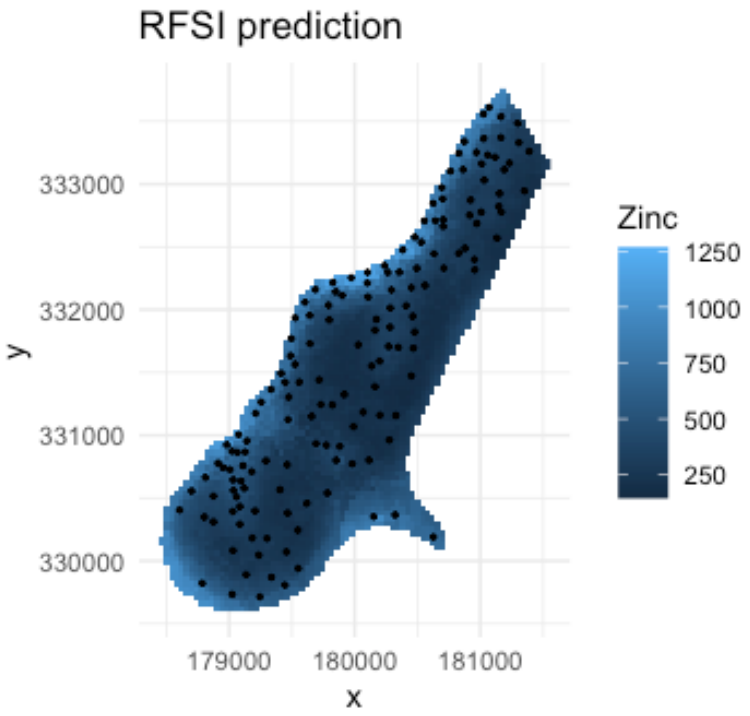
```
# Plot the Random Forest prediction map
ggplot(meuse.grid, aes(x = x, y = y, fill = zinc_RF)) +
  geom_raster() +
  geom_point(data = meuse, aes(x = x, y = y), inherit.aes = FALSE, size = 0.5)
+
  coord_equal() +
```

```
  theme_minimal() +
  labs(fill = "Zinc", title = "Random Forest prediction")
```



27. Plot the RFSI prediction map

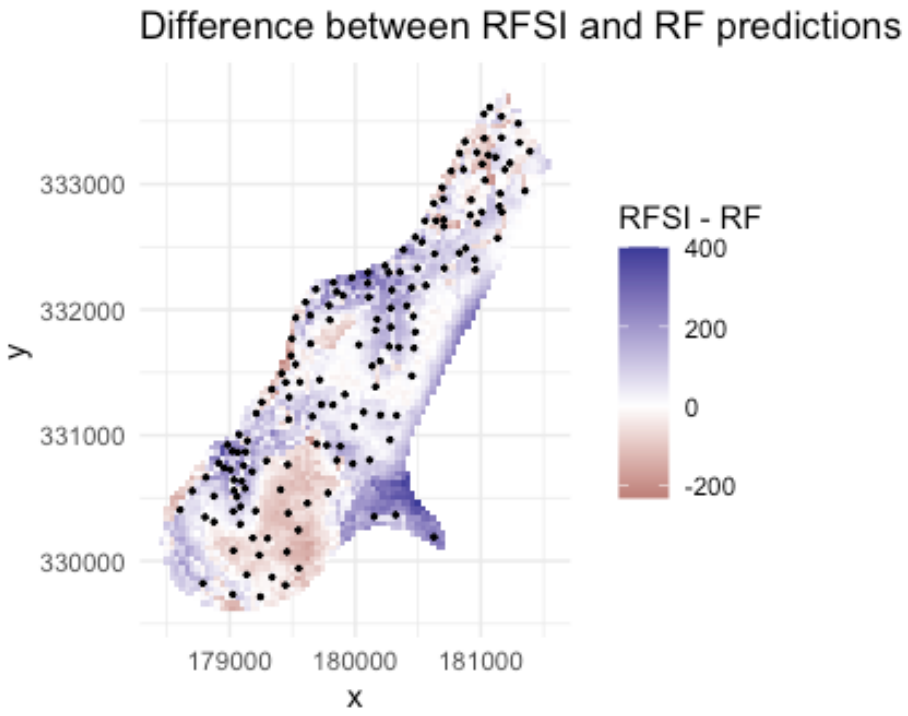
```
# Plot the RFSI prediction map
ggplot(meuse.grid, aes(x = x, y = y, fill = zinc_RFSI)) +
  geom_raster() +
  geom_point(data = meuse, aes(x = x, y = y), inherit.aes = FALSE, size = 0.5)
+
  coord_equal() +
  theme_minimal() +
  labs(fill = "Zinc", title = "RFSI prediction")
```



28. Plot the differences between RFSI and RF

This is probably one of the most important figures for the assignment.

```
# Plot the difference between RFSI and RF
ggplot(meuse.grid, aes(x = x, y = y, fill = diff)) +
  geom_raster() +
  geom_point(data = meuse, aes(x = x, y = y), inherit.aes = FALSE, size = 0.5)
+
  coord_equal() +
  scale_fill_gradient2(midpoint = 0) +
  theme_minimal() +
  labs(fill = "RFSI - RF", title = "Difference between RFSI and RF predictions")
```



This map allows us to identify the locations where explicitly incorporating spatial neighbourhood information changes the predicted zinc concentrations.

References

Sekulić, A., Kilibarda, M., Heuvelink, G. B. M., Nikolić, M., & Bajat, B. (2020). Random Forest Spatial Interpolation. Remote Sensing, 12(10), 1687. https://doi.org/10.3390/rs12101687

Hengl, T., Parente, L., & Bonannella, C. (2022). Spatial and Spatiotemporal Interpolation / Prediction using Ensemble Machine Learning (Version v0.1). Zenodo. https://doi.org/10.5281/zenodo.5894924